

Лабораторная работа № 2.1. Синтаксические деревья

18 февраля 2026 г.

Ольга Ивенкова, ИУ9-62Б

Цель работы

Целью данной работы является изучение представления синтаксических деревьев в памяти компилятора и приобретение навыков преобразования синтаксических деревьев.

Индивидуальный вариант

После выхода из любого цикла должно выводиться количество выполненных итераций.

Реализация

Демонстрационная программа:

```
package main

import "fmt"

func main() {
    for i := 0; i < 3; i++ {
        fmt.Println("for loop i:", i)
    }

    n := 0
    for n < 2 {
        fmt.Println("while-like n:", n)
        n++
    }

    for i := 0; i < 2; i++ {
```

```

        for j := 0; j < 2; j++ {
            fmt.Println("nested", i, j)
            if i == 1 && j == 1 {
                break
            }
        }
    }
}

```

Программа, осуществляющая преобразование синтаксического дерева:

```

package main

import (
    "fmt"
    "go/ast"
    "go/format"
    "go/parser"
    "go/token"
    "os"
)

var seq int

func nextCounter() string {
    seq++
    return fmt.Sprintf("__loopIter%d", seq)
}

func ensureFmtImported(f *ast.File) {
    for _, imp := range f.Imports {
        if imp.Path != nil && imp.Path.Value == "\"fmt\"" {
            return
        }
    }
    for _, d := range f.Decls {
        if g, ok := d.(*ast.GenDecl); ok && g.Tok == token.IMPORT {
            g.Specs = append(g.Specs,
                &ast.ImportSpec{Path: &ast.BasicLit{Kind: token.STRING, Value: "\"fmt\""}},
            )
            return
        }
    }
    f.Decls = append([]ast.Decl{
        &ast.GenDecl{
            Tok: token.IMPORT,
            Specs: []ast.Spec{
                &ast.ImportSpec{Path: &ast.BasicLit{Kind: token.STRING, Value: "\"fmt\""}},
            },
        },
    }, f.Decls...)
}

```

```

    },
    }, f.Decls...)
}

func prependInc(body *ast.BlockStmt, name string) {
    inc := &ast.IncDecStmt{X: ast.NewIdent(name), Tok: token.INC}
    if body == nil {
        body = &ast.BlockStmt{List: []ast.Stmt{inc}}
    } else {
        body.List = append([]ast.Stmt{inc}, body.List...)
    }
}

func processStmts(list []ast.Stmt) []ast.Stmt {
    out := make([]ast.Stmt, 0, len(list))
    for _, st := range list {
        switch s := st.(type) {
        case *ast.BlockStmt:
            s.List = processStmts(s.List)
            out = append(out, s)
        case *ast.IfStmt:
            if s.Body != nil {
                s.Body.List = processStmts(s.Body.List)
            }
            if s.Else != nil {
                if be, ok := s.Else.(*ast.BlockStmt); ok {
                    be.List = processStmts(be.List)
                } else if ei, ok := s.Else.(*ast.IfStmt); ok {
                    ei.Body.List = processStmts(ei.Body.List)
                }
            }
            out = append(out, s)
        case *ast.ForStmt:
            if s.Body != nil {
                s.Body.List = processStmts(s.Body.List)
            }
            name := nextCounter()
            prependInc(s.Body, name)
            init := &ast.AssignStmt{
                Lhs: []ast.Expr{ast.NewIdent(name)},
                Tok: token.DEFINE,
                Rhs: []ast.Expr{&ast.BasicLit{Kind: token.INT, Value: "0"}},
            }
            printCall := &ast.ExprStmt{
                X: &ast.CallExpr{

```

```

        Fun: &ast.SelectorExpr{X: ast.NewIdent("fmt"),
        Sel: ast.NewIdent("Printf")},
        Args: []ast.Expr{
            &ast.BasicLit{Kind: token.STRING,
                Value: `"iterations: %d\n"`},
            ast.NewIdent(name),
        },
    },
}
out = append(out, init, s, printCall)
default:
    out = append(out, st)
}
return out
}

func insertCounters(file *ast.File) {
    ensureFmtImported(file)
    for _, d := range file.Decls {
        if fn, ok := d.(*ast.FuncDecl); ok && fn.Body != nil {
            fn.Body.List = processStmts(fn.Body.List)
        }
    }
}

func main() {
    if len(os.Args) != 2 {
        fmt.Printf("usage: astprint <filename.go>\n")
        return
    }
    fset := token.NewFileSet()
    file, err := parser.ParseFile(
        fset,
        os.Args[1],
        nil,
        parser.ParseComments)
    if err != nil {
        fmt.Printf("Error: %v\n", err)
        return
    }

    insertCounters(file)

    if err := format.Node(os.Stdout, fset, file); err != nil {
        fmt.Printf("format error: %v\n", err)
    }
}

```

```
}  
}
```

Тестирование

Результат трансформации демонстрационной программы:

```
package main  
  
import "fmt"  
  
func main() {  
    __loopIter1 := 0  
    for i := 0; i < 3; i++ {  
        __loopIter1++  
        fmt.Println("for loop i:", i)  
    }  
    fmt.Printf("iterations: %d\n", __loopIter1)  
  
    n := 0  
    __loopIter2 := 0  
    for n < 2 {  
        __loopIter2++  
        fmt.Println("while-like n:", n)  
        n++  
    }  
    fmt.Printf("iterations: %d\n", __loopIter2)  
    __loopIter4 := 0  
  
    for i := 0; i < 2; i++ {  
        __loopIter4++  
        __loopIter3 := 0  
        for j := 0; j < 2; j++ {  
            __loopIter3++  
            fmt.Println("nested", i, j)  
            if i == 1 && j == 1 {  
                break  
            }  
        }  
        fmt.Printf("iterations: %d\n", __loopIter3)  
    }  
    fmt.Printf("iterations: %d\n", __loopIter4)  
}
```

Вывод тестового примера на stdout

```
for loop i: 0
for loop i: 1
for loop i: 2
iterations: 3
while-like n: 0
while-like n: 1
iterations: 2
nested 0 0
nested 0 1
iterations: 2
nested 1 0
nested 1 1
iterations: 2
iterations: 2
```

Вывод

Изучила представления синтаксических деревьев в памяти компилятора и приобрела навыки преобразования синтаксических деревьев.